

Efficient Java Exception Handling in Just-in-Time Compilation*

SeungIl Lee, Byung-Sun Yang,
Suhyun Kim, Seongbae Park,
Soo-Mook Moon
School of Electrical Engineering
Seoul National University
Seoul 151-742, Korea
{seungil, scdoner, zelo, ramdrive,
smoon}@altair.snu.ac.kr

Kemal Ebcioglu,
Erik Altman
IBM T. J. Watson Research Center
P.O. Box 218
Yorktown Heights, NY 10598
{kemal, erik}@us.ibm.com

ABSTRACT

The Java language provides *exceptions* in order to handle errors gracefully. However, the presence of exception handlers complicate the job of a JIT (Just-in-Time) compiler, including optimizations and register allocation, even though exceptions are rarely used in most programs.

This paper describes some mechanisms for removing overheads imposed by the existence of exception handlers, including *on-demand translation* of exception handlers, which expose more optimization opportunities in normal flow. In addition, we also minimize the exception handling overhead for frequently thrown exceptions by jumping directly from the exception throwing point into the exception handler through a technique called *exception handler prediction*.

Experiments show that the existence of exception handlers indeed does not interfere with the translation of normal flow using our exception handling mechanisms. Also, the results reveal that frequently thrown exceptions are efficiently handled with exception handler prediction.

1. INTRODUCTION

The Java programming language provides an exception handling mechanism for elegant error handling [5]. When an error occurs during execution of code in a *try* block, the error is caught and handled by an exception handler in one of the subsequent *catch* blocks associated with it. An optional *finally* block includes “cleanup” code which is always executed regardless of whether or not an error occurs in the *try* block. If no *catch* block can handle the error, the method

*This work has been supported by IBM T. J. Watson Research Center under a sponsored research agreement.

is terminated abnormally and the JVM (Java Virtual Machine) searches backward through the call stack to find an exception handler which can handle the error.

Exceptions are also useful for controlling program execution flow such as exiting nested loops or control transfers across call chains [3]. This is because of the feature that once an exception is generated during the execution of a method, the method is terminated and program control is transferred to an appropriate exception handler.

This paper describes the exception handling mechanism in the context of Java JIT compilation, which translates Java bytecode into native machine code at runtime, so that a Java program runs in the form of machine code without interpretation. JIT compilation provides better performance than interpretation, since it executes a program as machine code with various code optimizations applied [1, 4, 8, 9, 15]. However, the existence of exception handlers often prevents JIT compilers from aggressively optimizing code [6, 10, 14]. Our exception handling mechanism removes all such overheads so that most opportunities for optimizations may be exploited, and also improves the exception handling speed for frequently thrown exceptions using exception handler prediction.

The remainder of this paper is composed as follows. Section 2 shows several Java programs’ characteristics and motivations from them. Section 3 describes the way of achieving normal flow efficiency. Section 4 discusses about faster exception handling with exception handler prediction : a technique for statically identifying the handler for a given exception. Comparison to related works is given in Section 5. Section 6 presents experimental results. A summary follows in Section 7.

2. CHARACTERISTICS OF EXCEPTIONS IN JAVA PROGRAMS

Observing the characteristics of many Java programs motivated the key idea of our exception handling mechanisms. We present the properties of SPEC benchmarks that can be a representative set of Java programs [12].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Java 2000 San Francisco CA USA
Copyright ACM 2000 1-58113-288-3/00/6...\$5.00

Table 1: Usage of Exception Handlers in SPEC benchmarks

Benchmarks	Used EH	Total EH	ratio(%)
_200_check	24	79	30.3
_201_compress	0	45	0
_202_jess	0	53	0
_209_db	0	50	0
_213_javac	3	132	2.3
_222_mpegaudio	0	44	0
_227_mtrt	0	54	0
_228_jack	84	169	49.7

First, we found that few exception handlers are actually used in most programs. Table 1 represents how many exception handlers are actually used during program execution. 5 out of 8 SPEC benchmarks do not generate any exceptions, and thus exception handlers are left unused until the program ends. In the remaining 3 benchmarks, exception handlers are used, however there still exist unused exception handlers.

Table 2: Type Characteristics of Exceptions in SPEC Benchmarks

Benchmark	Generated Exceptions	Unique Type	Multi Type	ratio (%)
_200_check	25	25	0	100
_213_javac	22,372	22,372	0	100
_228_jack	241,876	241,876	0	100

Another notable thing is that the exception type is unique at a certain throwing point through the program execution when the program uses exceptions, although it is possible that the type can vary. That is mainly because exceptions are used for program flow control, and not for error handling in these programs. Table 2 shows the type characteristics of exceptions thrown at a certain point in SPEC benchmarks. As can be seen from Table 2, all throwing points throw only a single type, not multiple types.

3. EFFICIENCY ON NORMAL FLOW

As shown in Section 2, exception handlers are seldom used in most programs, i.e., they belong to a method only for potential errors, and are not actually needed in most cases. However, most JIT compilers restrict code optimizations on normal flow if exception handlers exist [10, 14]. We propose methods to remove such inefficiencies due to the existence of exception handlers.

3.1 On-demand Translation of Exception Handlers

So far most JIT compilers translate exception handlers when translating a method [1, 10, 14]. Instead we determined not to translate an exception handler together with the other parts of a method, because we know that an exception is unlikely to be used.

The on-demand translation of exception handlers has two major benefits, one of which is to reduce JIT compilation time, and the other is to simplify the control flow of a method. The JIT compilation time can be saved, because JIT compilation time is normally dependent on bytecode size and exception handlers of the method will be excluded from the targets of its translation. Also, the control flow is

simplified since exception handling flow does not interfere with normal flow. As a result, JIT compiler can generate better code for normal flow compared to when the exception handling code is translated together.

3.2 Allowing More Optimizations

Finding exception handlers is basically done using the exception table attached to a method in class files. The exception table is an array of the quadruple (**start_pc**, **end_pc**, **handler_pc**, **catch_type**), where an exception of **catch_type** raised between **start_pc** and **end_pc** in the program is handled by a handler whose start address is **handler_pc** [11]. All these pc's are bytecode addresses.

After a method is translated into native code by a JIT compiler, only native addresses are available in the translated code. Thus the JVM should be able to find exception handlers based on native addresses. One solution to this problem is to check the address range similarly to interpreters, replacing bytecode addresses with corresponding native addresses in the exception table. In order to make the range check possible, JIT compiler must preserve the original basic block order in the translated code, which interferes with aggressive code optimizations [10, 14].

Another solution, used in our exception handling scheme, creates another table called the *exception information table* which is a hash table that contains (**native address**, **bytecode pc**) pairs at all potential exception raising instructions in try blocks. By looking up this table with the native address as a key, the original bytecode pc can be retrieved, which then can be used for checking with the original exception table. This approach provides an advantage that we do not have to maintain the original structure of the bytecode during JIT compilation, allowing more aggressive optimizations, including code motion across basic blocks.

Figure 1 shows how effectively our JIT compiler can generate the normal flow code with our exception information table. The basic blocks connected by **goto** instruction is merged into one basic block, and loop invariant instructions are moved out of the loop, which is possible with the help of the exception information table. Note that the generated code is the same as when there is no catch block.

3.3 Efficient Handling of Finally Blocks

Although the Java programming language is type safe, which means all local variables are guaranteed to have unique types at a certain point, there exists one exception related to **finally** blocks. **Finally** blocks, which are also called as a subroutine at the bytecode level, are defined with the **finally** keyword and handled with the '**jsr**' bytecode. If a JVM encounters '**jsr**' bytecode while executing, it stores the current bytecode pc onto the operand stack and jumps to the target bytecode pc, and it might be able to return using the stored bytecode pc. The JVM specification permits a local variable to have different types in a subroutine depending on the path on which the subroutine is called [11]. For example, the type of local variable 1 is ambiguous in the subroutine of Figure 2. The variable is an integer if the subroutine is called from normal code, and a reference if called from exception handling code.

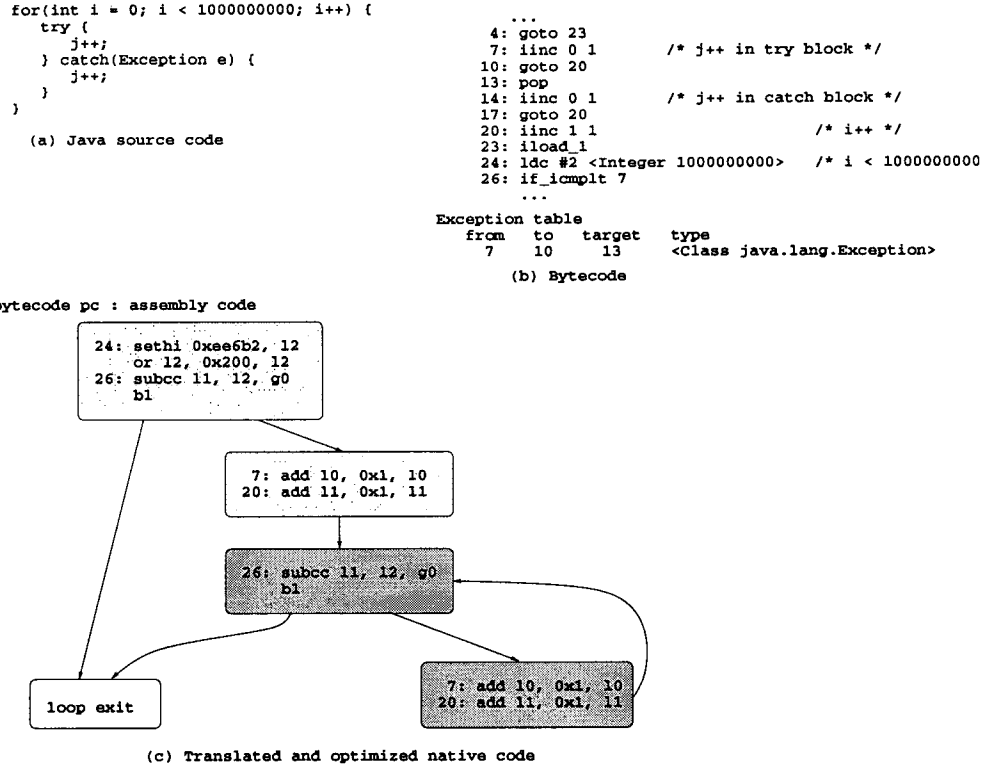


Figure 1: Basic block ordering example

This type ambiguity may cause complications or restrictions on JIT compilation or garbage collection. The register allocator in a JIT compiler should know the type of a local variable in order to map it onto an appropriate register. The garbage collector should also know whether a local variable is a reference or not when garbage collection occurs for precise detection of live objects [13].

In order to remove the type ambiguity, we may split conflicting local variables by bytecode rewriting [2]. However, this splitting requires data flow analysis and bytecode rewriting, which can be expensive.

Thus, we choose to inline a subroutine into every `jsr` bytecode. If a subroutine is inlined to a path, then the types of the local variables become fixed for that path, so that the safety hole is removed. The one concern is that it might cause code expansion since the subroutine is duplicated for every `jsrs`. However, in most cases there exist one `jsr` in normal code and others in exception handling code. Hence if exception handlers are not translated in our on-demand translation scheme, subroutines are inlined only once, so we do not have to worry about code expansion.

Figure 2 illustrates how the `jsr` problem is solved by inlining subroutines. The ambiguous type of local variable 1 is now fixed as an integer on the normal flow path and as a reference on the exception handling flow after inlining finally blocks. Moreover, since exception handling flow is generated only after exceptions are generated with the on-demand translation scheme, the code of exception handling flow, including duplicated inlined subroutine may not be

generated, and only one copy of the subroutine may exist if no exception for the exception handler is generated during program execution.

4. FASTER EXCEPTION HANDLING

So far, we have assumed exceptions are rare in Java programs. However, we found that exceptions are used frequently as a tool controlling program execution flow in some programs, as we have seen in Section 2. And we expect that exceptions will be used more extensively as the Java programming language becomes popular. Therefore fast exception handling is as important as improving translation of normal flow. In order to handle generated exception, the JVM must search an exception handler based on the exception type and transfer control to the handler, which requires saving and restoration of program context. In this section, we will introduce a way to minimize this overhead by eliminating JVM intervention.

4.1 Exception Handler Prediction

Exceptions used frequently are thrown by a user with a `'throw'` statement in most cases. The `'throw'` statement requires an exception object whose type is used for finding an appropriate exception handler. Normally, the exception object is created at the `throw` statement or a variable which stores the exception object is used in the `'throw'` statement as shown in Figure 3. In such cases, exceptions thrown at a certain point will have only one type.

Therefore, we can predict that the exception handler used for the exception thrown at a point will be reused the next

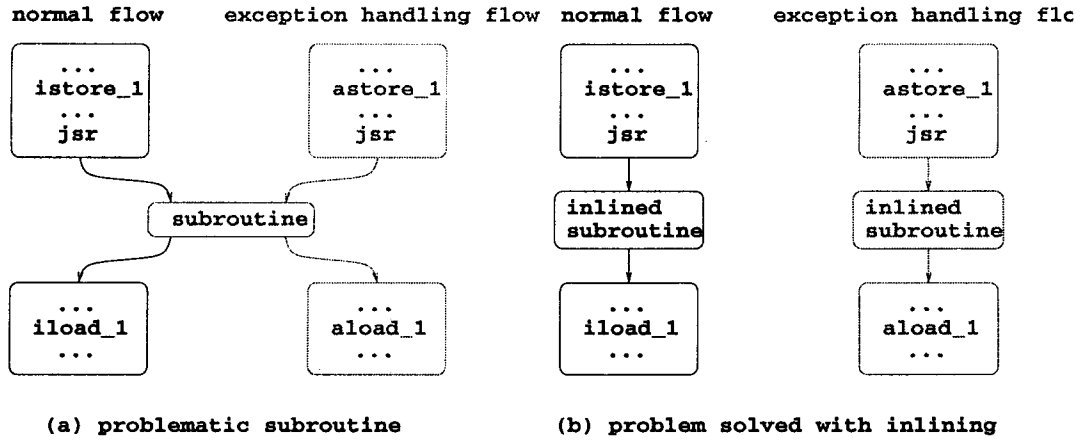


Figure 2: Jsrr problem solved with inlining subroutine

```

try {
    ...
    throw new MyException();
} catch(MyException e) {
    ...
}

MyException eobj = new
MyException();
...
try {
    ...
    throw eobj;
} catch(MyException e) {
    ...
}

```

Figure 3: Typical usage of exceptions

time. Since the same exception type is expected for the next time, the trap to the routine in the JVM that finds the exception handler can be simply replaced with a direct jump to the exception handler, once an appropriate exception handler is found. With this simple conversion, the exception handler searching process will be bypassed next time.

This simple conversion is only possible for the case where the exception throwing method and catching method are identical, since when the methods are different, the prediction becomes too complicated because the runtime call chain may vary.

However, this simple conversion is not enough to deal with all possible cases. One problem is that it is not quite certain that the same type of the exception will be generated at the same throwing point unless the exception usage follows the usage in Figure 3. The other problem is that local variables defined previously during the execution of normal flow can be used in the exception handler, so that the values of local variables should be passed into the exception handlers.

In order to solve the first problem, it is necessary to check the exception type before the direct jump. A data flow analysis might be used for completely removing this check, although it is not implemented in our current work. If the exception type is not matched, the normal exception handling process should be invoked. For the second problem, the location of local variables at the exception handler entry is adjusted

for the first invoked exception, so that no additional value passing sequences are needed. When the translated exception handler is used from other point afterwards, some copy instructions that propagates values of local variables into the handlers may be necessary before the exception handler entry.

Figure 4 presents the result code generated between the throwing point and the predicted exception handler. Only a few machine instructions are executed if the exception object has an appropriate type for the exception handler. Since insertion of this connection code does not modify existing normal flow code, the code quality of normal flow is still preserved.

4.2 Extension of Exception Handler Prediction with Method Inlining

Exception handler prediction has the limitation that the throwing and catching methods should be the same method. However, this limitation can be partially removed through method inlining done by the JIT compiler. When the throwing and catching methods are different, they can be merged into one method, and the exception handler prediction is still applicable in this case, because the runtime call chain is fixed after method inlining.

Figure 5 shows a part of a call graph extracted from the `_228-jack` SPEC benchmark. The `scan_token` method throws an exception, and it is caught by one of the methods `_Jack2.44` and `_Jack2.43` according to the call chain. After method inlining is applied, the exception generated at each throwing point is now caught by a unique exception handler, and the exception handler can be directly invoked from the throwing point with exception handler prediction.

5. RELATED WORK

Other JVMs like CACAO or Kaffe keep the original basic block ordering and give up aggressive optimizations when applying JIT compilation on methods with exception handlers, since their exception handler finding process is based on address range check [10, 14]. In contrast to them, our exception handling mechanism can alter basic block ordering because we have an additional address translation table

```

// fetch handler type constant into r2 register
ld    [%exception_object%], r1    // get type info from exception object
cmp    r1, r2                    // compare types of the object and catch block
be     _matched_case
call   exception_manager          // non-matched case normal handling
_matched_case:
... resolving codes ...           // some copy instructions
call   translated_catch_code      // matched case fast handling

```

Figure 4: Pseudo code of type checking in front of exception handler

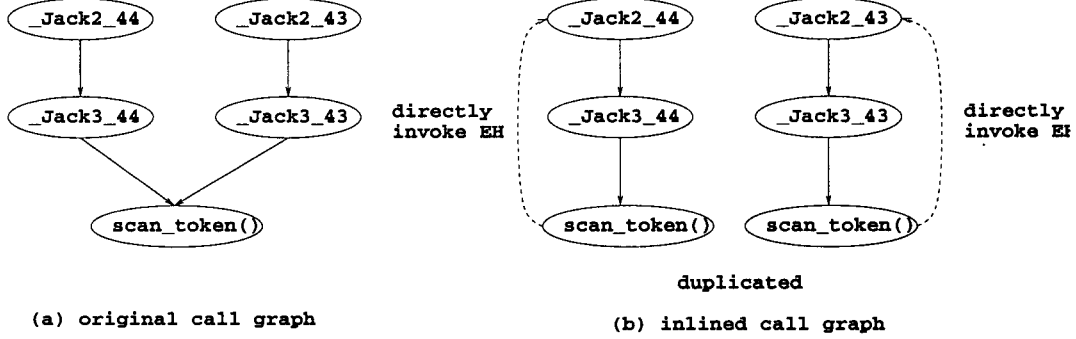


Figure 5: Direct connection with method inlining

built during JIT compilation. We can remove the restriction by sacrificing memory, and do more optimizations on normal flow.

The idea of delaying translation of exception handlers until actual usage is similar with the `uncommon traps` in the SELF compiler in the aspect of delaying compilation of infrequent paths [7]. The uncommon trap in the SELF compiler inserts a call to the compilation process at the start of an infrequent path instead of compiled code, and after compilation the call to the compiler is replaced by a jump to the compiled code. However exception handling raises more complicated problem. In our on-demand translation, which exception handler is required is not determined until the exception is generated, sometimes rewinding the call stack is required and subroutines must be dealt with correctly. With our exception handler prediction, we connect exception throwing points and exception handlers after finding exception handlers. With the uncommon trap in SELF, after the infrequent path is compiled, the original execution point is connected to the compiled code. The two ideas look similar in the sense of connecting two paths. However, our exception handler prediction requires more than just transferring program control; an exception type check is necessary between the connection, and the prediction can be interprocedural through method inlining.

6. EXPERIMENTAL RESULTS

The exception handling mechanism described in the previous sections has been implemented on the *LaTTe* Java virtual machine, which does aggressive register allocation along with some classical optimizations such as common subexpression elimination, loop invariant code motion, and objective oriented optimizations such as method specialization [15].

We compared the exception handling overhead of *LaTTe* with the SUN JDK 1.1.7 production release (*SUN117*), and a version of *LaTTe* in which the JIT compiler is replaced with Kaffe's JIT compiler (*Kaffe*) to show the efficiency of our exception handling mechanism. All of the compared JVMs are equipped with JIT compilers.

In order to check the effectiveness of exception handler prediction, we made three variants of *LaTTe*, and compared them with the original one. The first version lacks exception handler prediction, the second version is without the method inlining, and the third version has neither.

All experiments were performed on a SUN UltraSPARC III 270MHz machine running Solaris 2.6 with 256 MB of memory, and the minimum execution time is taken among 5 runs.

6.1 Micro Benchmarks

In order to figure out the impact of the existence of exception handling structures, we tested the two programs that are shown in Figure 6. The first program is a normal program with no exception handling structure, while the other one is similar program with `try`, `catch`, and `finally` blocks. Both programs do not generate any exceptions, and thus behave identically.

Table 3: Comparison for non-exception generating programs

Benchmark	Execution Time (sec)		
	LaTTe	SUN117	Kaffe
BaseCase	10.3	22.6	71.4
NoException	10.3	49.4	175.9
Overhead	0	26.8	104.5

Table 3 compares *LaTTe* with the other JVMs. As can be

```

public class Base {
    public static
    void main(String[] argv) {
        int j = 0;
        for(int i = 0; i < 1000000000; i++) {
            j++;
            j++;
        }
    }
}

```

```

public class NoException {
    public static
    void main(String[] argv) {
        int j = 0;
        for(int i = 0; i < 1000000000; i++) {
            try {
                j++;
            } catch(Exception e) {
                j++;
            } finally {
                j++;
            }
        }
    }
}

```

Figure 6: Test programs in which no exception occurs

seen from Table 3, there is little difference between the two programs of Figure 6 with our exception handling scheme, while the other JVMs show significant differences, which means that the existence of `try`, `catch` and `finally` blocks in a method degrades performance even though no exception occurs during execution.

To measure the effectiveness of our exception handler prediction, we tested the two programs shown in Figure 7.¹ In these programs, the same exception is generated in the loop repeatedly. The two programs differ only in the existence of an intervening method between the throwing and catching methods.

Table 4 shows the results of testing the programs shown in Figure 7. As can be seen in Table 4, exception handler prediction effectively reduces exception handling time, and also direct connection was made possible across call chains after method inlining.

6.2 Performance Impact on Real Benchmarks

Table 5: Methods with Exception Handlers in SPEC Benchmarks

Benchmark	Number of Methods	Methods with EH	Ratio (%)
_200_check	265	49	18.49
_201_compress	209	32	15.31
_202_jess	576	37	6.42
_209_db	334	37	16.59
_213_javac	905	84	9.28
_222_mpegaudio	316	31	9.81
_227_mtrt	286	39	13.64
_228_jack	430	99	23.02
GEOMEAN			13.09

Table 5 presents how many methods have exception handlers in the SPEC benchmarks [12]. As shown in Table 5, about 13 % of all methods have exception handlers. Our exception handling mechanism can affect translation of these methods.

Table 6 shows how much of the bytecode is actually translated when on-demand translation is employed. The remaining bytecode are unused exception handlers which are not

¹These types of structures are used for measuring exception handling speed in one of the Java Grande benchmarks.

Table 6: Actual Translation of Bytecode in SPEC Benchmarks

Benchmark	Translated Bytecode	Total Bytecode	Ratio (%)
_200_check	25,506	26,551	96.06
_201_compress	22,340	23,220	96.21
_202_jess	41,685	43,179	96.54
_209_db	24,108	25,219	95.59
_213_javac	82,963	89,470	92.73
_222_mpegaudio	36,129	37,018	97.60
_227_mtrt	30,810	32,108	95.96
_228_jack	47,777	49,916	95.71
GEOMEAN			95.79

translated. We can estimate the JIT compilation time is reduced by approximately 4 % due to the on-demand translation scheme, because JIT compilation time is approximately proportional to bytecode size [15].

Table 7: Handler prediction effect in _228_jack benchmark

	Elapsed Time(sec)			
	Original	Modified	Prediction	Ratio
Total	50.68	60.54	57.13	1.13
TR	8.09	19.13	19.12	2.36
Pure	42.59	41.40	38.01	0.89

Number of Finding Exception Handlers		
Original	Prediction	Ratio
241,876	67,238	0.27

Exception handler prediction has no effect on the execution time of SPEC benchmarks, since the throwing and catching methods are not inlined into one method with the default inlining heuristic, since the heuristic only depends on the bytecode size. To make the exception handler prediction effective, inlining heuristic should be modified, which causes JIT compilation time to increase significantly as shown in Table 7. Although the translation time takes significant portion, the pure execution time is reduced by 11 % with exception handler prediction due to the decreased number of finding process. It is left as future work to find out better inlining heuristics that exploits exception handler prediction and which requires little addition JIT overhead.

```

class NoCallChain {
    public static
    void main(String[] argv) {
        int j = 0;
        for(int i = 0; i < 1000000; i++) {
            try {
                j++;
                throw new Exception();
            } catch(Exception e) {
                j++;
            }
        }
    }
}

```

```

class CallChain {
    public static
    void throw_method() throws Exception {
        throw new Exception();
    }

    public static
    void main(String[] argv) {
        int j = 0;
        for(int i = 0; i < 1000000000; i++) {
            try {
                j++;
                throw_method();
            } catch(Exception e) {
                j++;
            }
        }
    }
}

```

Figure 7: Test programs in which exceptions occur frequently

Table 4: Comparison of exception generating programs

Benchmark	Execution Time (sec)					
	LaTTe				SUN117	Kaffe
	neither	no connect	no inline	both		
NoCallChain	24.0	17.8	14.1	8.9	22.3	14.0
CallChain	28.4	18.9	27.9	9.3	23.4	16.1

7. SUMMARY

In this paper, we have described the exception handling mechanisms used in our LaTTe JIT compiler, which are on-demand exception handler translation and exception handler prediction.

Our exception handling mechanisms remove overheads imposed on normal flow by the existence of exception handlers, and also effectively work for frequently thrown exceptions with exception handler prediction.

Currently exceptions are not popularly used. However, they are a clean and useful feature for easy programming and we expect that they will be used more and more in the future, so that an efficient exception handling scheme such as ours will become essential.

8. REFERENCES

- [1] A.-R. Adl-Tabatabai, M. Cierniak, G.-Y. Lueh, V. M. Parikh, and J. M. Stichnoth. Fast, effective code generation in a just-in-time Java compiler. In *Proceedings of the ACM SIGPLAN '98 Conference on Programming Language Design and Implementation*, pages 280–290, 1998.
- [2] O. Agesen, D. Detlefs, and J. E. B. Moss. Garbage collection and local variable type-precision and liveness in Java virtual machines. In *Proceedings of the ACM SIGPLAN '98 Conference on Programming Language Design and Implementation (PLDI)*, pages 269–279, Montreal, Canada, 17–19 June 1998.
- [3] K. Arnold and J. Gosling. *The Java Programming Language*. Addison Wesley, 1997.
- [4] K. Ebcioglu, E. Altman, and E. Hokenek. A JAVA ILP machine based on fast dynamic compilation. In *MASCOTS '97 - International Workshop on Security and Efficiency Aspects of Java*, 1997.
- [5] J. Gosling, B. Joy, and G. Steele. *The Java Language Specification*. Addison Wesley, 1997.
- [6] M. Gupta, J.-D. Choi, and M. Hind. Optimizing Java programs in the presence of exceptions. In *Proceedings of the 14th European Conference on Object-Oriented Programming (ECOOP'00)*, 2000.
- [7] U. Holzle. Adaptive optimization for SELF: Reconciling high performance with exploratory programming. Thesi CS-TR-94-1520, Stanford University, Department of Computer Science, Aug. 1994.
- [8] A. Krall. Efficient JavaVM just-in-time compilation. In J.-L. Gaudiot, editor, *International Conference on Parallel Architectures and Compilation Techniques*, pages 205–212, Paris, Oct. 1998. IFIP,ACM,IEEE, North-Holland.
- [9] A. Krall and R. Grafl. CACAO – A 64 bit JavaVM just-in-time compiler. In G. C. Fox and W. Li, editors, *PPoPP'97 Workshop on Java for Science and Engineering Computation*, Las Vegas, June 1997. ACM.

- [10] A. Krall and M. Probst. Monitors and exceptions: How to implement Java efficiently. In S. Hassanzadeh and K. Schauser, editors, *ACM 1998 Workshop on Java for High-Performance Computing*, pages 15–24, Palo Alto, Mar. 1998. ACM.
- [11] T. Lindholm and F. Yellin. *The Java Virtual Machine Specification*. Addison-Wesley, Reading, USA, 1997.
- [12] SPEC JVM98 benchmarks.
<http://www.spec.org/osg/jvm98/>, 1998.
- [13] J. M. Stichnoth, G.-Y. Lueh, and M. Cierniak. Support for garbage collection at every instruction in a Java compiler. In *Proceedings of SIGPLAN'99 Conference on Programming Languages Design and Implementation*, ACM SIGPLAN Notices, pages 118–127, Atlanta, May 1999. ACM Press.
- [14] T. Wilkinson. Kaffe: A JIT and interpreting virtual machine to run Java code.
<http://www.transvirtual.com/>, 1998.
- [15] B.-S. Yang, S.-M. Moon, S. Park, J. Lee, S. Lee, J. Park, Y. C. Chung, S. Kim, K. Ebcioglu, and E. Altman. LaTTe: A Java VM just-in-time compiler with fast and efficient register allocation. In *Proceedings of the 1999 International Conference on Parallel Architectures and Compilation Techniques*, Oct. 1999.